Eloquent ORM

Use models to read and save database records. Replace the example model and field names with the ones already provided in your project.

Pattern	Meaning
`Book::all()`	Get every book as a collection.
`Book::findOrFail($id)`	Get one book by ID, or return HTTP 404.
`Book::where('available', true)->get()`	Filter rows, then run the query.
`Book::orderBy('title')->get()`	Sort books by title.
`Book::create($validated)`	Insert a row and return the new model.
`$book->update($validated)`	Save changes to an existing row.
`LibraryLoan::whereDate('return_due_at', today()->addDay())->orderBy('return_due_at')->get()`	Get loans due on the next calendar day, ordered by return time.

Remember: build the query first; `get()` runs it and returns a collection. `whereDate()` compares only the date, so different times on that day still match.

For `create()` and `update()`, the model must allow the fields through `$fillable`. Workshop models are already configured.

Request validation

Validate input before saving. For a web form, Laravel sends errors back to the page when a rule fails.

Rule or pattern	Meaning
`required`	The field must be present and nonempty.
`string`	The value must be text.
`numeric / integer`	Accept a number / a whole number.
`min:10 / max:255`	With string: character limits. With numeric: number limits.
`between:1,8`	Accept values from 1 to 8, including both endpoints.
`nullable`	Allow null for an optional field.
`in:small,medium,large`	Accept only one of the listed values.
`'quantity' => [ 'required', 'integer', 'between:1,8', ]`	Combine rules in the existing validation array. Here, quantity must be a whole number from 1 to 8.
`$validated = $request->validate($rules);`	Return the accepted data, or stop and report errors.

Remember: use `numeric` or `integer` for numeric bounds. Use `string` when checking text length.

Save validated data after the check. Assign server-owned fields, such as an initial status, in the controller rather than trusting the submitted value.

Authentication & authorization

Authentication identifies the user. Authorization decides which actions that user may perform.

Pattern	Meaning
`auth()->check()`	Is someone signed in?
`auth()->user()`	Get the signed-in User, or null for a guest.
`$user->role === 'archivist'`	Check whether the role exactly matches archivist.
`Gate::allows('read-notes')`	Return true or false for the current user.
`Gate::authorize('read-notes')`	Continue when allowed; otherwise deny access, normally with HTTP 403.
`Gate::define('read-notes', function (User $user): bool { return $user->role === 'archivist'; } );`	Define a named permission in AppServiceProvider::boot(). Laravel supplies the authenticated user to the callback.

Remember: use === to compare values; = assigns a value. In a non-nullable User callback like this, guests are denied automatically.

Enforce the Gate on the server before retrieving protected data. A hidden button does not protect a route. Check both an allowed identity and a denied identity.

These patterns use `App\Models\User` and `Illuminate\Support\Facades\Gate`. The workshop already supplies the surrounding code.

PHP decisions & commands

A match expression maps an input to an output. Compare exact values, separate arms with commas, and keep a fallback.

Pattern	Meaning
`$label = match ($service) { 'express' => 'Next day', 'pickup' => 'Collect in store', default => 'Standard delivery', };`	The input express returns Next day. The result is assigned to $label, ready to display or save.

Remember: `match` uses strict comparisons. Input spelling and capitalization must match the form values.

Daily commands

Run these in the project folder. Use the Ubuntu terminal in WSL or Terminal on macOS; keep Docker Desktop running.

Command	Purpose
`sh workshop.sh`	Start or resume the app.
`sh workshop.sh test BusTest`	Run one test class; replace BusTest as needed.
`sh workshop.sh seed`	Refresh sample dates.
`sh workshop.sh stop`	Stop the app; keep saved data.
`php artisan test --filter=BusTest`	Same test, using a native PHP setup instead of Docker.

Test classes: `AdvisorTest, BankingTest, BusTest, CityTest, NewsroomTest`.

Windows PowerShell outside WSL:

```
powershell -NoProfile -ExecutionPolicy Bypass
-File .\workshop.ps1 test BusTest
```

Enter that PowerShell command on one line. To start the app instead, omit `test BusTest`.